

Guía paso a paso · Despliegue F1 CMS

Manual operativo del despliegue del CMS **CSM** en AWS EC2 conectado al MySQL del cloud SDS (Francia), con la web pública del campeonato F1 2025 alimentada por el dataset toUpperCase78/formula1-datasets.

Grupo: Edgar · Álvaro · Samuel · José **Repositorio CMS:** <https://github.com/FomoDonkey/TechX>
Dataset F1: <https://github.com/toUpperCase78/formula1-datasets>

Mapa de máquinas

Emoji	Máquina	IP / Acceso	Quién corre qué
🟢	Local (tu Windows)	tu portátil	Solo <code>ssh</code> y navegador
🟠	EC2 AWS	<code><IP_EC2></code> (la verás en consola AWS)	Docker + Caddy + git + npm
🔵	SDS Francia	<code>5.196.50.3:33060</code>	Solo MySQL — entras desde la EC2
🟣	Container csm	dentro de la EC2	<code>docker compose exec csm</code> <code><cmd></code>

Importante: no necesitas instalar nada en tu Windows local salvo un cliente SSH. Todos los comandos `mysql`, `docker`, `npm`, `openssl` los ejecutas en la EC2 (es Ubuntu y ya trae casi todo).

FASE 1 — 🟠 Lanzar la EC2 en AWS

1.1 En la consola web de AWS

1. Entra a `console.aws.amazon.com` → servicio **EC2**.
2. Pulsa **Launch instance**.
3. Configuración recomendada:
 - **Name:** `csm-f1`
 - **AMI:** Ubuntu Server 24.04 LTS (HVM)

- **Instance type:** `t3.small` (2 vCPU, 2 GB RAM) — `t2.micro` se queda corto al hacer `next build`
- **Key pair:** crea uno nuevo `csm-f1-key.pem` y descárgalo a `C:\Users\edgar\.ssh\csm-f1-key.pem`
- **Network settings** → **Allow:** SSH (22), HTTP (80), HTTPS (443)
- **Storage:** 16 GB gp3 (suficiente)

4. Launch instance.

1.2 Asociar Elastic IP (importante)

Sin Elastic IP, si paras y arrancas la EC2 cambia la IP pública → tienes que rehacer el grant del MySQL cada vez. Con Elastic IP la IP queda fija.

1. Consola EC2 → **Elastic IPs** → **Allocate Elastic IP address** → confirmar.
2. Selecciona la nueva IP → **Actions** → **Associate** → elige tu instancia `csm-f1`.

1.3 Anotar la IP pública

Anótala — la vas a usar en todos los pasos siguientes:

```
<IP_EC2> = X.X.X.X ← tu Elastic IP
```

FASE 2 — Conectar desde tu Windows a la EC2

Abre **PowerShell** en tu Windows local:

```
#  [Local Windows]
ssh -i C:\Users\edgar\.ssh\csm-f1-key.pem ubuntu@<IP_EC2>
```

Primera conexión te dirá "*The authenticity of host... can't be established*" → escribe `yes`.

Si funciona, verás algo como:

```
Welcome to Ubuntu 24.04.1 LTS
ubuntu@ip-172-31-XX-XX:~$
```

Desde aquí, todo lo que escribas hasta hacer `exit` se ejecuta dentro de la EC2.

Si te dice "Permissions are too open" en el .pem (Windows lo da con permisos amplios):

```
# [Local Windows PowerShell]
icacls C:\Users\edgar\.ssh\csm-f1-key.pem /inheritance:r /grant:r "$($env:USERNAME):(R)"
```

FASE 3 — Preparar la EC2

3.1 Actualizar paquetes e instalar herramientas base

```
# [AWS EC2]
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl ufw mysql-client openssl
```

`mysql-client` te permitirá conectar al SDS desde la EC2 sin instalar nada en tu Windows.

3.2 Configurar firewall UFW

```
# [AWS EC2]
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp comment 'SSH'
sudo ufw allow 80/tcp comment 'HTTP'
sudo ufw allow 443/tcp comment 'HTTPS'
sudo ufw --force enable
sudo ufw status
```

3.3 Instalar Docker

```
# [AWS EC2]
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
newgrp docker
docker --version          # verificar
docker compose version    # verificar
sudo systemctl enable docker
```

3.4 Instalar Node.js 24

```
# [AWS EC2]
curl -fsSL https://deb.nodesource.com/setup_24.x | sudo -E bash -
sudo apt install -y nodejs
node --version          # debe decir v24.x.x
npm --version
```

FASE 4 — Generar las contraseñas y secretos

Genera **3 valores aleatorios** y guárdalos en un sitio seguro (gestor de contraseñas, papel, lo que sea):

```
# [AWS EC2] — ejecuta los 3 comandos y APUNTA cada salida

# Password A — la del usuario MySQL 'csm'
echo "MYSQL_CSM_PASS=$(openssl rand -hex 24)"

# Password B — opcional, solo si rotas el root del SDS
echo "MYSQL_ROOT_PASS=$(openssl rand -hex 24)"

# AUTH_SECRET — secret de sesiones + cifrado AI keys
echo "AUTH_SECRET=$(openssl rand -hex 32)"
```

Vas a usar:

- **MYSQL_CSM_PASS** en la fase 5 (**CREATE USER**) y en la fase 6 (**DATABASE_URL**).
- **AUTH_SECRET** en la fase 6 (**.env**).

Uso **-hex** en lugar de **-base64** para evitar caracteres especiales (**+** , **/** , **=**) que romperían la URL del **DATABASE_URL**.

FASE 5 — Preparar la base de datos en el SDS

Sigues conectado a la EC2 — desde aquí lanzas el cliente MySQL apuntando al SDS Francia.

5.1 Verificar conectividad EC2 → SDS

```
# [AWS EC2] → conecta a [SDS Francia]
mysql -h 5.196.50.3 -P 33060 -u root -p -e "SELECT VERSION();" 
```

Te pide la **password root del MySQL del SDS** (esa la tiene tu grupo de cuando levantasteis el container MySQL).

Si responde con la versión (algo como **8.4.x**), buen estado. Si **timeout** → revisa firewall del SDS (debe permitir tu IP de EC2 en el puerto 33060).

5.2 Crear la base de datos y el usuario CSM

```
# [AWS EC2] → conecta a [SDS Francia]
mysql -h 5.196.50.3 -P 33060 -u root -p
```

Te pide la password root otra vez. Cuando entres al prompt **mysql>**, pega este SQL **sustituyendo <IP_EC2> por tu Elastic IP de la fase 1.3 y <MYSQL_CSM_PASS> por la password A de la fase 4:**

```
-- [SDS Francia] – dentro del prompt mysql>

CREATE DATABASE IF NOT EXISTS csm_f1
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

DROP USER IF EXISTS 'csm'@'%';
DROP USER IF EXISTS 'csm'@'<IP_EC2>';

CREATE USER 'csm'@'<IP_EC2>' IDENTIFIED BY '<MYSQL_CSM_PASS>';
GRANT ALL PRIVILEGES ON csm_f1.* TO 'csm'@'<IP_EC2>';
FLUSH PRIVILEGES;

SELECT User, Host FROM mysql.user WHERE User = 'csm';
EXIT;
```

Salida esperada:

```
+-----+-----+
| User | Host |
+-----+-----+
| csm  | X.X.X.X | ← tu IP_EC2
+-----+-----+
```

5.3 Verificar que el usuario csm puede conectar

```
# [AWS EC2] → conecta a [SDS Francia] como usuario csm
mysql -h 5.196.50.3 -P 33060 -u csm -p csm_f1 -e "SELECT 1;"
```

Te pide MYSQL_CSM_PASS. Si responde **1**, perfecto. Si dice **Host is not allowed to connect** → tu IP_EC2 no está bien en el grant: revisa fase 5.2.

FASE 6 — Clonar y configurar el CMS

6.1 Clonar el repo

```
#  [AWS EC2]
cd ~
git clone https://github.com/FomoDonkey/TechX.git csm
cd csm
```

6.2 Crear el `.env` con tus valores

```
#  [AWS EC2]
cp .env.docker.example .env
nano .env
```

Edita estas **5 líneas** (deja todo lo demás por defecto). Sustituye `<MYSQL_CSM_PASS>` y `<AUTH_SECRET>` por los valores de la fase 4:

```
# Imprescindibles para F1:
DATABASE_URL=mysql://csm:<MYSQL_CSM_PASS>@5.196.50.3:33060/csm_f1
MYSQL_VECTOR_FALLBACK=true
AUTH_SECRET=<AUTH_SECRET>
NEXT_PUBLIC_APP_URL=http://<IP_EC2>:3000
APP_HOST_BIND=0.0.0.0      # provisional, hasta tener Caddy (FASE 11)
```

Guardar en nano: `Ctrl+O`, `Enter`, `Ctrl+X`.

6.3 Permisos restrictivos al `.env`

```
#  [AWS EC2]
chmod 600 ~/csm/.env
ls -l ~/csm/.env      # debe mostrar -rw-----
```

FASE 7 — Cargar el dataset F1

```
#  [AWS EC2] — desde ~/csm
npm run f1:setup
```

El script:

1. Clona `toUpperCase78/formula1-datasets` en `data/formula1/`.

2. Ejecuta `sync-f1-data.mjs` para regenerar `src/blocks/spectacular/f1-data.ts`.
3. Imprime resumen tipo:

```
Drivers 2025: 21
Teams 2025: 10
Calendar 2025: 24 GPs
...
✓ F1 dataset listo
```

FASE 8 — Levantar el CMS

```
#  [AWS EC2] - desde ~/csm
docker compose -f docker-compose.yml \
               -f docker-compose.external-db.yml build csm

docker compose -f docker-compose.yml \
               -f docker-compose.external-db.yml up -d csm
```

El primer build tarda **5-10 minutos** (instala deps, hace `next build`).

8.1 Ver logs y esperar a "Ready"

```
#  [AWS EC2]
docker compose -f docker-compose.yml \
               -f docker-compose.external-db.yml logs -f csm
```

Espera a ver:

```
[csm] Esperando a la base de datos...
[csm] BD lista.
[csm] Aplicando schema...
[csm] Arrancando server Next.js...
▲ Next.js 15.x.x
- Local:      http://0.0.0.0:3000
✓ Ready in 1234 ms
```

`Ctrl+C` para salir de los logs (no para el container).

8.2 Alias para no escribir tanto

```
# [AWS EC2]
echo "alias dccf1='docker compose -f docker-compose.yml -f docker-compose.external-db.yml'" >> ~/.
source ~/.bashrc
```

A partir de aquí puedes usar `dccf1 up -d csm`, `dccf1 logs -f csm`, `dccf1 exec csm sh`, etc.

FASE 9 — Sembrar usuario admin y workspace

```
# [AWS EC2] → [Container csm]
dccf1 exec csm npm run db:seed
```

Salida esperada:

```
🌱 Seed CSM (idempotente)
✓ Workspace creado: demo
✓ Colecciones builtin (posts, pages)
✓ Branch main creada
✓ Usuario demo creado: demo@csm.dev / pwd: demo1234
✓ Usuario asignado como owner del workspace demo
✅ Seed completo
```

FASE 10 — Verificar en navegador y aplicar plantilla F1

10.1 Abrir el CMS

Desde tu Windows, abre el navegador:

```
http://<IP_EC2>:3000
```

Deberías ver la landing pública de CSM.

10.2 Login

```
http://<IP_EC2>:3000/login
```

Credenciales del seed:

- Email: `demo@csm.dev`
- Password: `demo1234`

10.3 Cambiar la password (importante)

Una vez dentro: `/admin/perfil` → **Cambiar contraseña**. La `demo1234` solo es de arranque.

10.4 Aplicar plantilla F1

1. Navega a `/admin/plantillas`.
2. Filtro categoría **Deportes**.
3. Localiza la card "**F1 Grand Prix — Inmersivo**" → click → preview espectacular.
4. Pulsa "**Usar esta plantilla**".
5. Cambia el título a `Campeonato F1 2025` → **Publicar**.

10.5 Marcar como home

1. `/admin/paginas` → fila de la página F1 → **Establecer como home**.
2. Abre `http://<IP_EC2>:3000/` → ahora carga la web F1 espectacular.

FASE 11 — HTTPS con Caddy (recomendado)

Solo si tienes un **dominio apuntando a** `<IP_EC2>` (registro DNS tipo A). Si no, salta esta fase.

11.1 Instalar Caddy en la EC2

```
#  [AWS EC2]
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/gpg.key | \
  sudo tee /etc/apt/trusted.gpg.d/caddy-stable.asc
curl -fsSL https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt | \
  sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy
```

11.2 Configurar Caddyfile

```
#  [AWS EC2]
sudo nano /etc/caddy/Caddyfile
```

Sustituye `<DOMINIO_EC2>` por tu dominio real:

```
<DOMINIO_EC2> {  
  encode gzip zstd  
  reverse_proxy 127.0.0.1:3000 {  
    header_up X-Forwarded-Proto https  
    transport http {  
      read_timeout 300s  
    }  
  }  
}  
log {  
  output file /var/log/caddy/access.log  
  format json  
}
```

11.3 Cambiar bind de Docker a localhost

```
# [AWS EC2]  
nano ~/csm/.env
```

Cambia:

```
APP_HOST_BIND=127.0.0.1  
NEXT_PUBLIC_APP_URL=https://<DOMINIO_EC2>
```

```
# [AWS EC2]  
dccf1 up -d csm # reinicia con la nueva config  
sudo systemctl reload caddy
```

11.4 Verificar

```
# [AWS EC2]  
curl -I https://<DOMINIO_EC2>  
# Debe responder HTTP/2 200
```

Caddy obtiene Let's Encrypt automáticamente. Tarda 30-60s la primera vez.

Checklist final

- ☐ EC2 con Elastic IP fija

- ☐ MySQL del SDS responde con `mysql -h 5.196.50.3 -P 33060 -u csm -p csm_f1 -e "SELECT 1;"`
- ☐ `docker ps` muestra el container `csm` Up X minutes (healthy)
- ☐ `http://<IP_EC2>:3000` carga la home pública
- ☐ Login en `/admin` funciona
- ☐ Página F1 publicada y marcada como home
- ☐ (Opcional) HTTPS funciona en `https://<DOMINIO_EC2>`

Comandos de operación diaria

```
# 🟠 [AWS EC2] – todos asumen alias dccf1 ya creado

dccf1 logs -f csm           # ver logs en vivo
dccf1 restart csm           # reiniciar
dccf1 exec csm sh           # entrar al container
dccf1 exec csm npm run db:seed # reseed (idempotente)
dccf1 ps                   # estado
docker stats csm-csm-1      # CPU/RAM del container

# Actualizar a último commit del repo:
cd ~/csm && git pull && dccf1 build csm && dccf1 up -d csm
```

Errores comunes y solución rápida

Error	Causa	Fix
<code>ECONNREFUSED</code> al hacer <code>db:push</code>	EC2 no llega al MySQL del SDS	Revisa fase 5.1 + firewall del SDS
<code>Host is not allowed to connect</code>	grant SQL no coincide con IP_EC2	Repite fase 5.2 con la IP correcta
<code>Unknown data type: 'VECTOR'</code>	MySQL 8.x sin fallback	Confirma <code>MYSQL_VECTOR_FALLBACK=true</code> en <code>.env</code> y <code>dccf1 restart csm</code>
Caddy: <code>bind: address already in use</code>	Apache/Nginx ocupando 80/443	<code>sudo systemctl stop apache2 nginx && sudo systemctl disable apache2 nginx</code>
El container reinicia en bucle	Error en <code>.env</code>	<code>dccf1 logs csm tail -50</code> , busca el error, edita <code>.env</code> , <code>dccf1 up -d csm</code>
Olvidaste guardar <code>AUTH_SECRET</code>	No hay forma de recuperarlo	Generas uno nuevo y haces <code>db:seed</code> otra vez (perderás sesiones y AI keys cifradas, pero no los datos)

Variables de entorno (resumen)

```
# OBLIGATORIAS
DATABASE_URL=mysql://csm:<PASS>@5.196.50.3:33060/csm_f1
AUTH_SECRET=<64 hex chars de openssl rand -hex 32>
MYSQL_VECTOR_FALLBACK=true           # solo MySQL 8.x

# RED
APP_HOST_BIND=127.0.0.1               # 0.0.0.0 antes de Caddy, 127.0.0.1 con Caddy
APP_PORT=3000
NEXT_PUBLIC_APP_URL=https://<DOMINIO> # o http://<IP_EC2>:3000 sin Caddy

# OPCIONALES (todas desactivan su feature si no se definen)
RESEND_API_KEY=                       # email transaccional
ANTHROPIC_API_KEY=                    # AI Agente Editorial
OPENAI_API_KEY=
GROQ_API_KEY=
STRIPE_SECRET_KEY=                    # memberships (no requerido para F1)
BLOB_READ_WRITE_TOKEN=                # uploads cloud
CRON_SECRET=                           # cron externo
```

Fin de la guía. Si algún paso falla, consulta la sección **Errores comunes** o el documento extenso [docs/PROYECTO-F1-CMS-DEPLOY.md](#).